L11a

February 26, 2026

1 Data Visualization with Matplotlib & Pandas

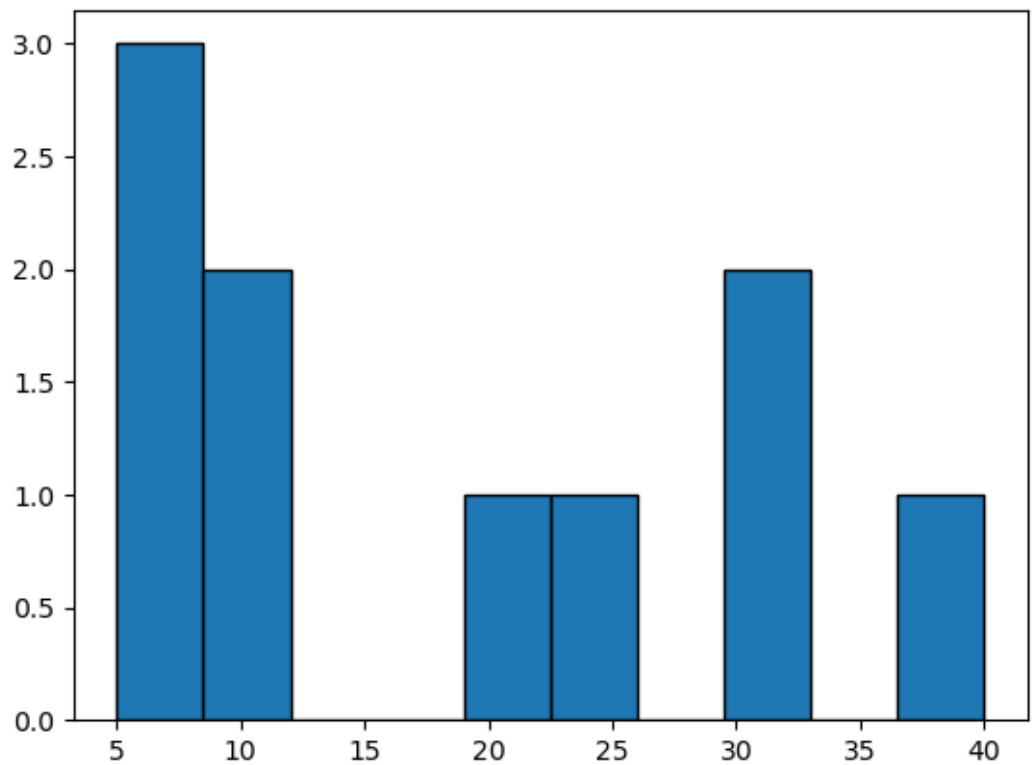
This guide covers essential plotting techniques for data science students, ranging from basic distribution plots to 3D visualization and Pandas integration.

1.1 1. Basic Plotting with Matplotlib

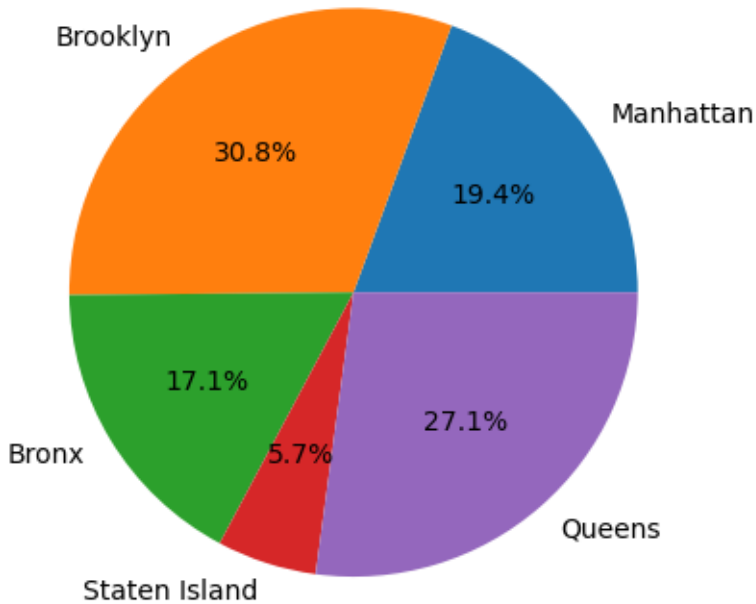
First, we import the library and set up the inline magic command for Jupyter.

[1]:
```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
%matplotlib inline
```

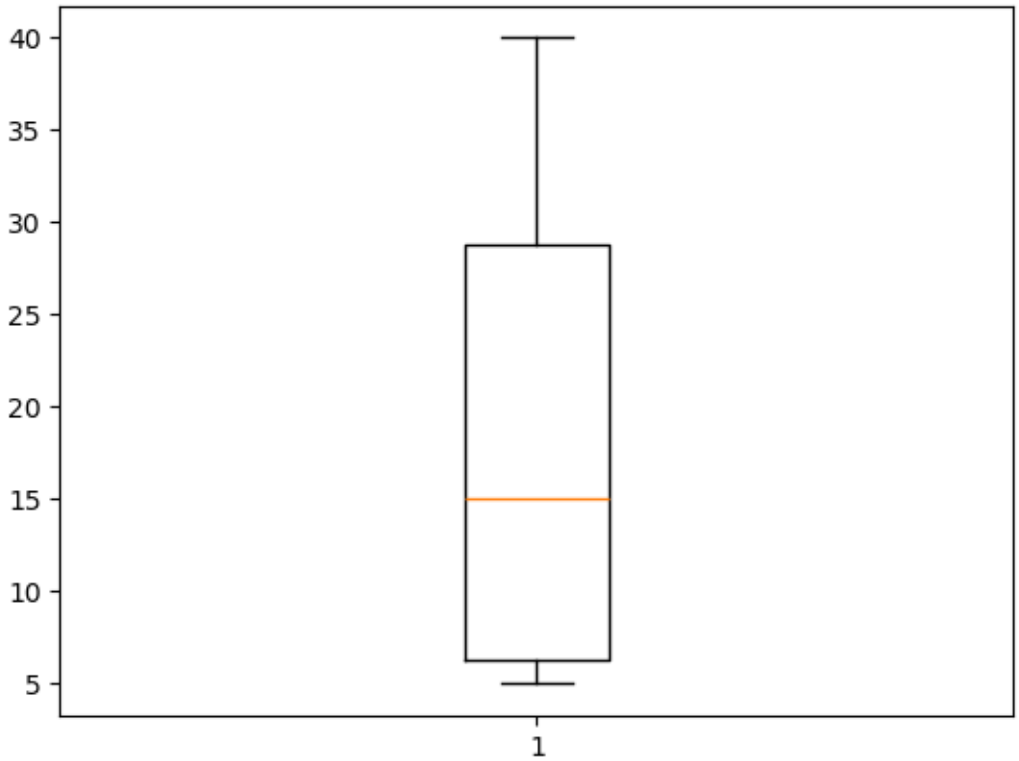
[2]:
```
# Histogram (Frequency distribution)
x = [5,5,5,10,10,30,30,20,25,40]
plt.hist(x, edgecolor='black');
```



[3]:
```
# Pie Chart (Parts of a whole)
values = [1628701 ,2582830, 1432132, 476179 , 2278906]
labels = ['Manhattan','Brooklyn','Bronx','Staten Island', 'Queens']
plt.pie(values, labels=labels, autopct='%1.1f%%');
```



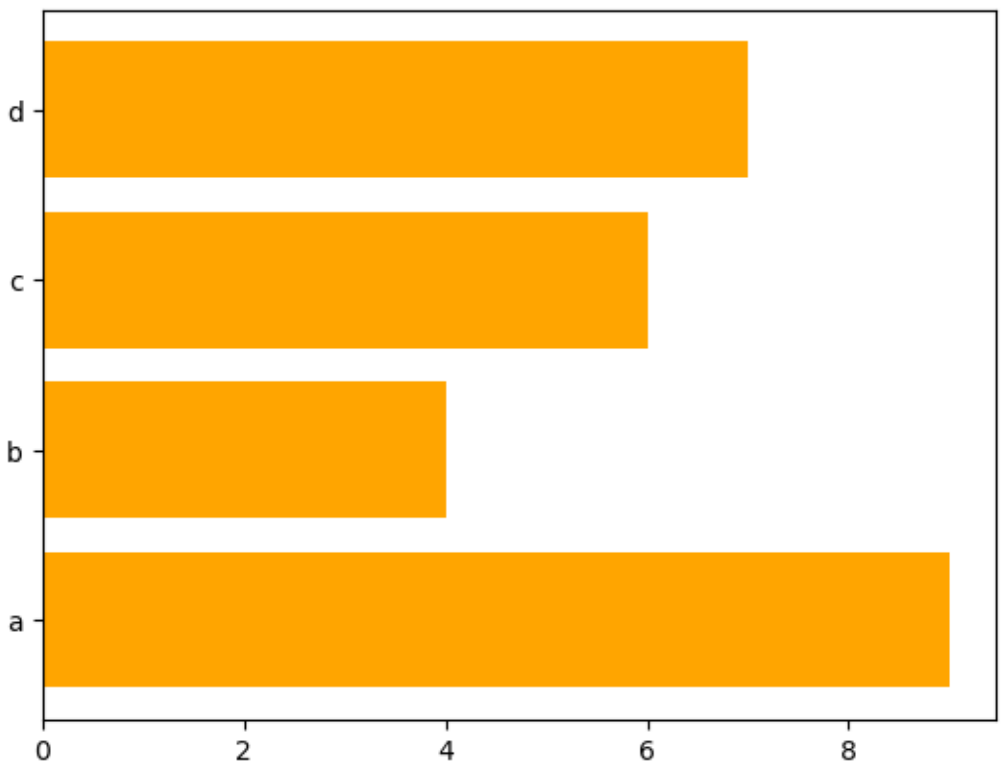
[4]:
```
# Boxplot (Distribution & Outliers)
# Shows Q1, Q2 (Median), Q3, and Whisker limits
plt.boxplot(x);
```



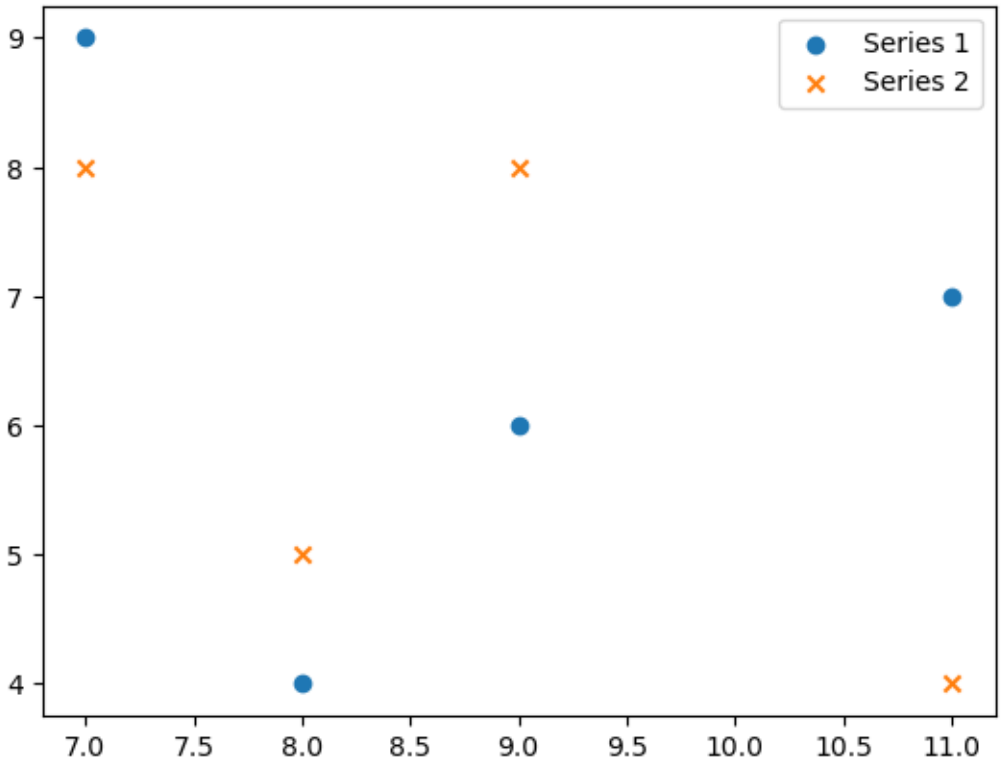
1.2 2. Categorical and Comparison Plots

Using bar charts and scatter plots to compare different variables.

[6]:
```
# Bar and Horizontal Bar Charts
categories = ['a','b','c','d']
counts = [9,4,6,7]
# plt.bar(categories, counts);
plt.barh(categories, counts, color='orange');
```



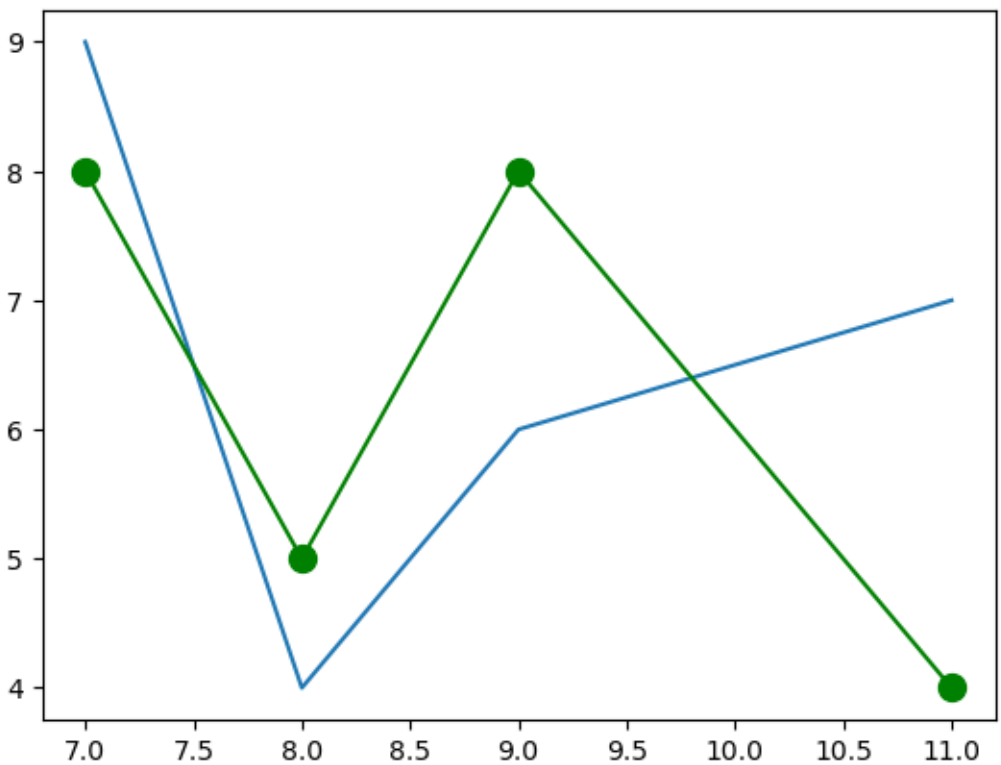
[13]:
```
# Scatter Plots
x_scat = [7,8,9,11]
y_scat = [9,4,6,7]
y2_scat = [8,5,8,4]
plt.scatter(x_scat, y_scat, label='Series 1')
plt.scatter(x_scat, y2_scat, label='Series 2', marker='x')
plt.legend();
```



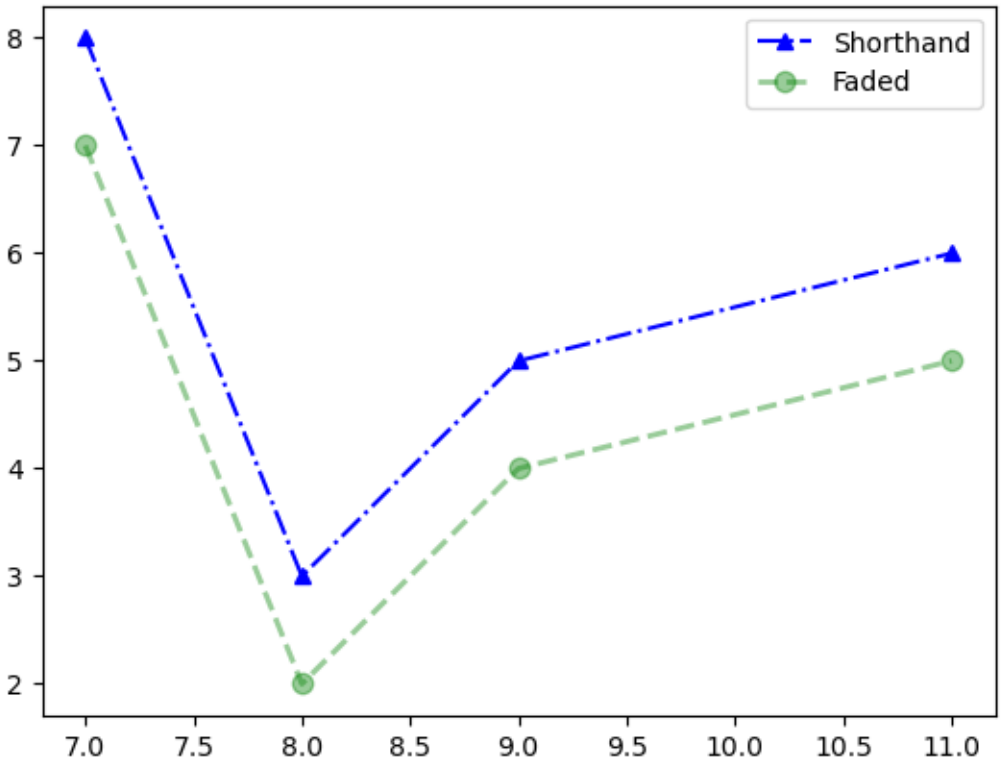
1.3 3. Line Plots and Styling

Customizing markers, line styles, and colors.

[17]:
```
x_line = [7,8,9,11]
y_line = [9,4,6,7]
# Standard Plot
plt.plot(x_line, y_line)
plt.plot(x_line, y2_scat, color='green', marker='o', markersize=10,␣
↪label='Basic');
```



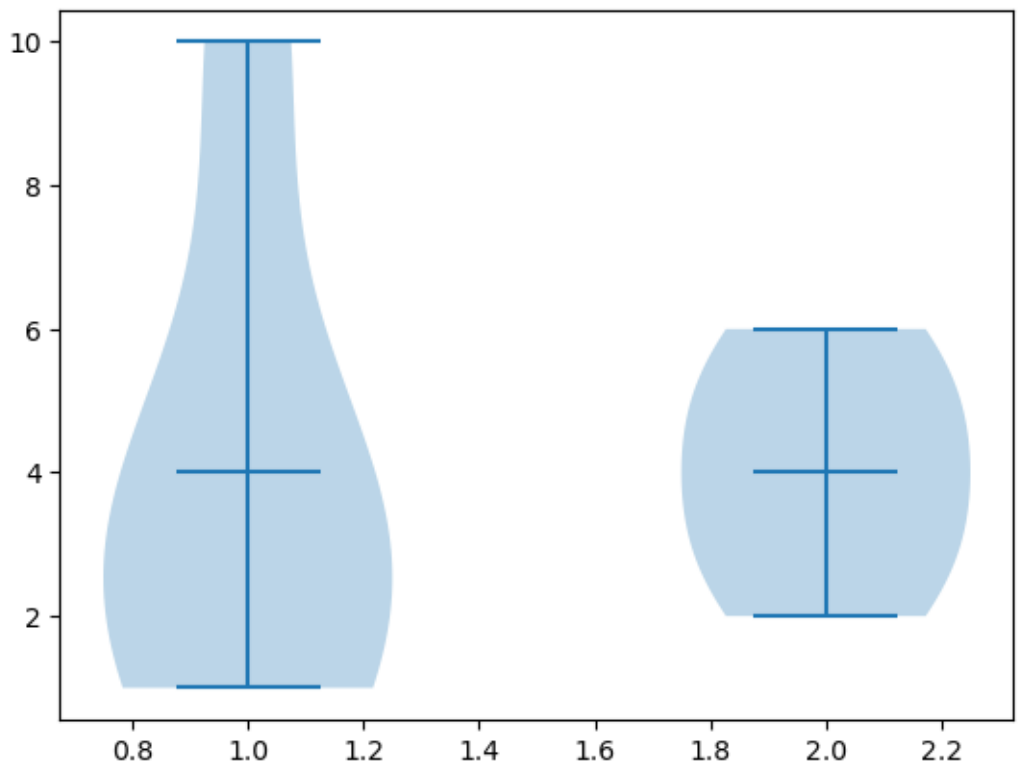
[18]:
```
# Shorthand notation (color, marker, linestyle)
plt.plot(x_line, [i-1 for i in y_line], 'b^-.', label='Shorthand')

# Transparency and line width
plt.plot(x_line, [i-2 for i in y_line], 'g.--', markersize=15, alpha=0.4,␣
 ↪linewidth=2, label='Faded')
plt.legend();
```



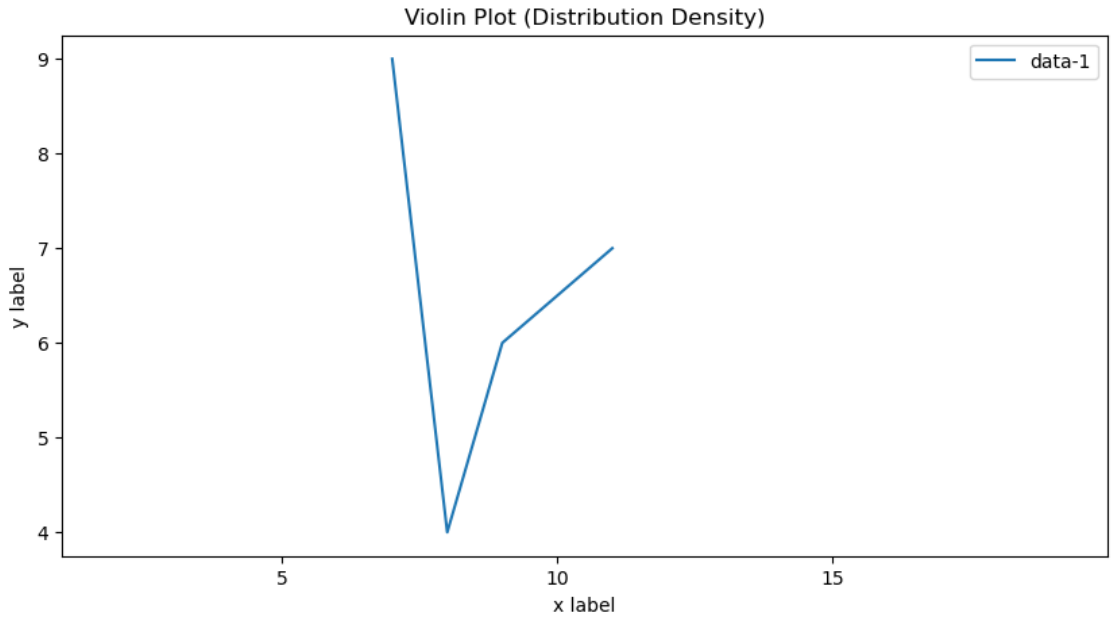
1.4 4. Advanced Charts

Visualizing statistical distributions and multidimensional data.

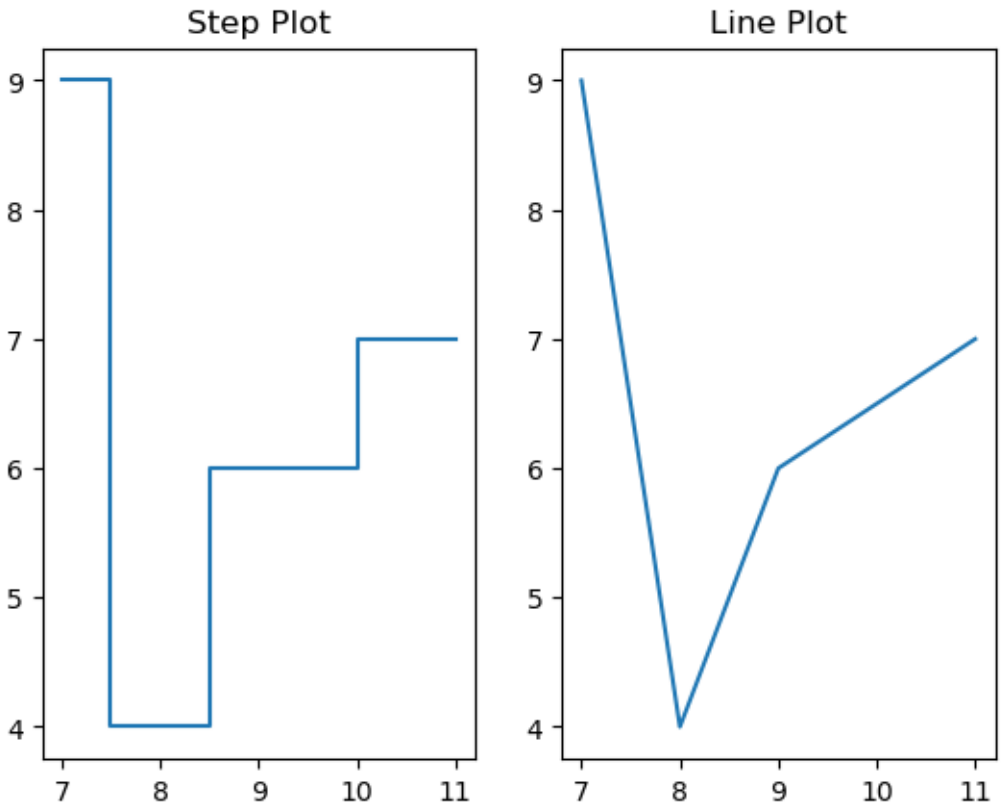
[19]:
```
# Step and Violin Plots
data_v = [[1, 2, 3, 4, 10], [2, 3, 4, 5, 6]]
plt.violinplot(data_v, showmeans=True);
```



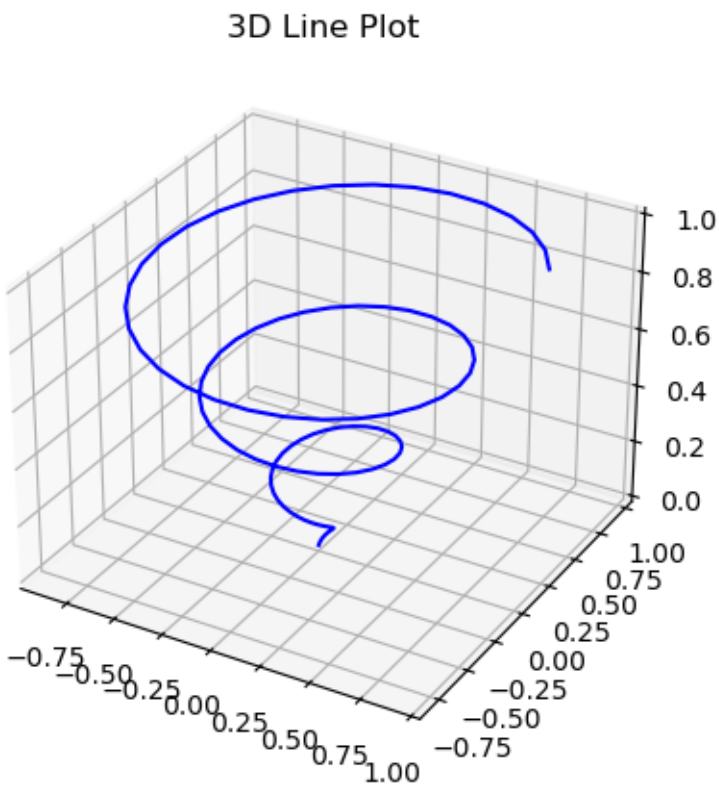
[24]:
```
plt.figure(figsize=(10,5))
plt.plot(x_line,y_line, label='data-1')
plt.title('Violin Plot (Distribution Density)')
plt.xlabel('x label')
plt.ylabel('y label')
plt.xlim([1,20])
plt.xticks([5,10,15])
plt.legend(loc=1)
plt.show()
```



[27]:
```
# Subplots
fig, ax = plt.subplots(1, 2)
ax[0].step(x_line, y_line, where='mid')
ax[0].set_title('Step Plot')
ax[1].plot(x_line, y_line)
ax[1].set_title('Line Plot')
plt.show()
```



[28]:
```
# 3D Plotting
ax_3d = plt.axes(projection="3d")
z = np.linspace(0, 1, 100)
x_3d = z * np.sin(20 * z)
y_3d = z * np.cos(20 * z)
ax_3d.plot3D(x_3d, y_3d, z, 'blue')
ax_3d.set_title('3D Line Plot')
plt.show()
```



[ ]:

```

```

1.5 5. Visualization within Pandas

[29]:

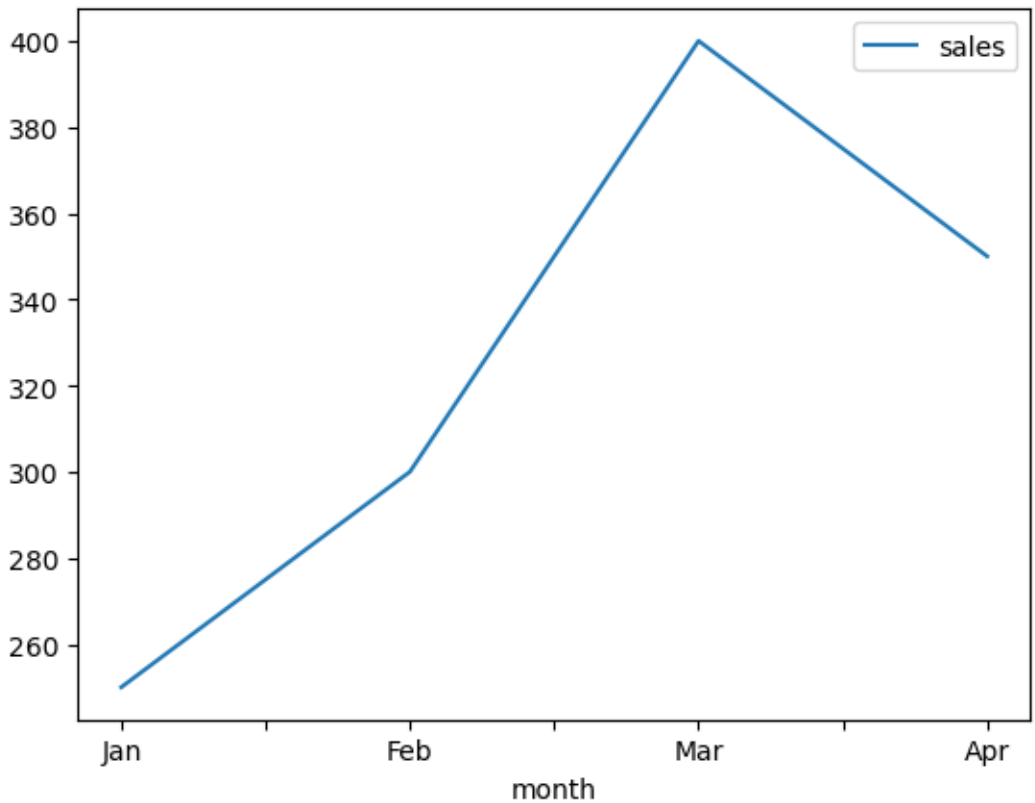
```
data={ 'month':['Jan','Feb', 'Mar', 'Apr'],
       'sales':[250,300,400,350],
       'expenses':[200,220,250,300]}
df=pd.DataFrame(data)
df
```

[29]:

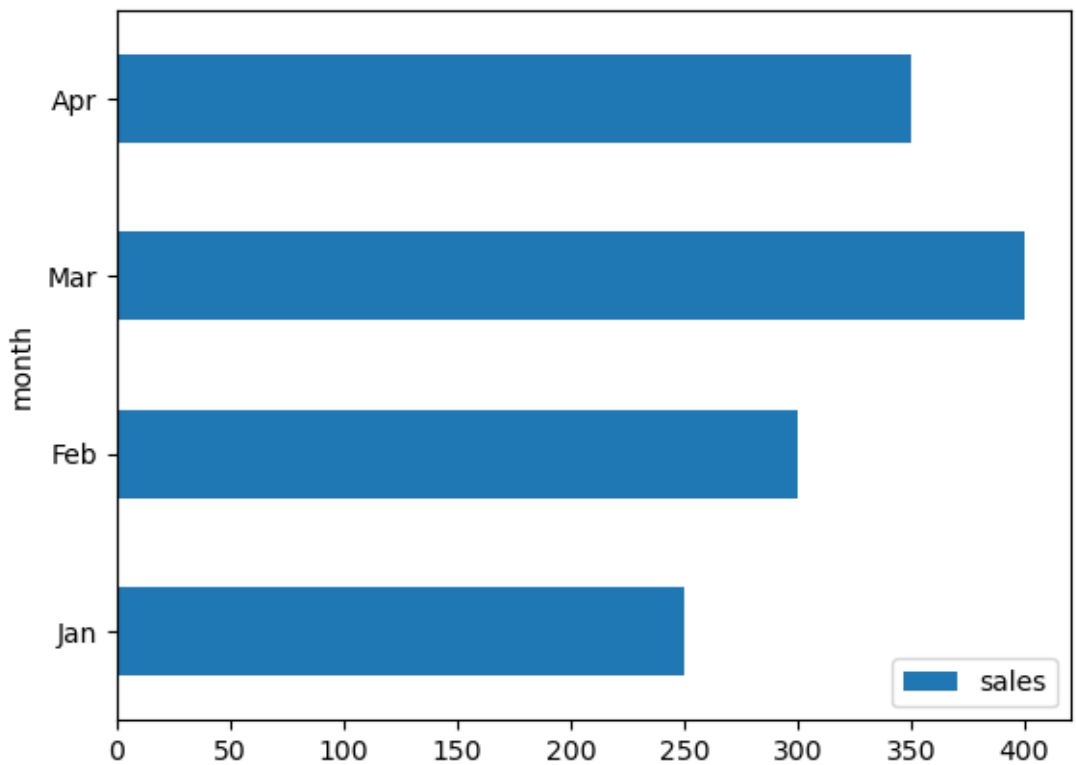
	month	sales	expenses
0	Jan	250	200
1	Feb	300	220
2	Mar	400	250
3	Apr	350	300

[31]:

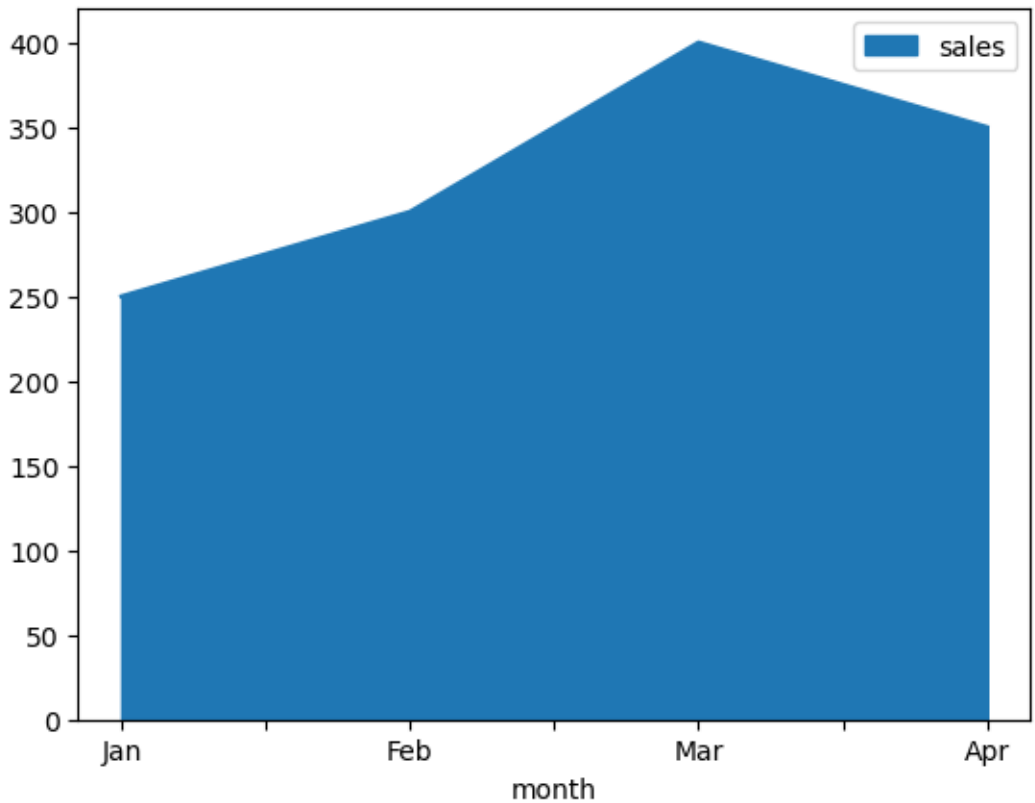
```
# line plot
# plt.plot(df.month, df.sales);
df.plot(x='month', y='sales');
```



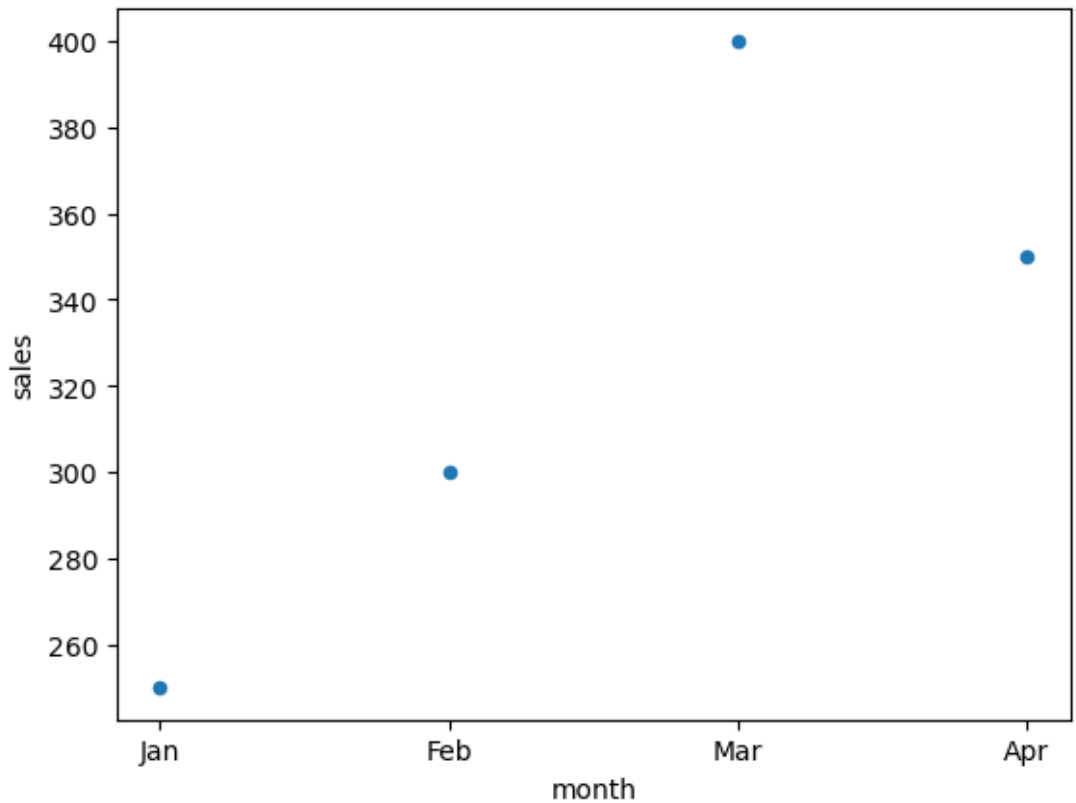
[33]:
```
# bar plot
# df.plot(x='month', y='sales', kind='bar');
df.plot(x='month', y='sales', kind='barh');
```



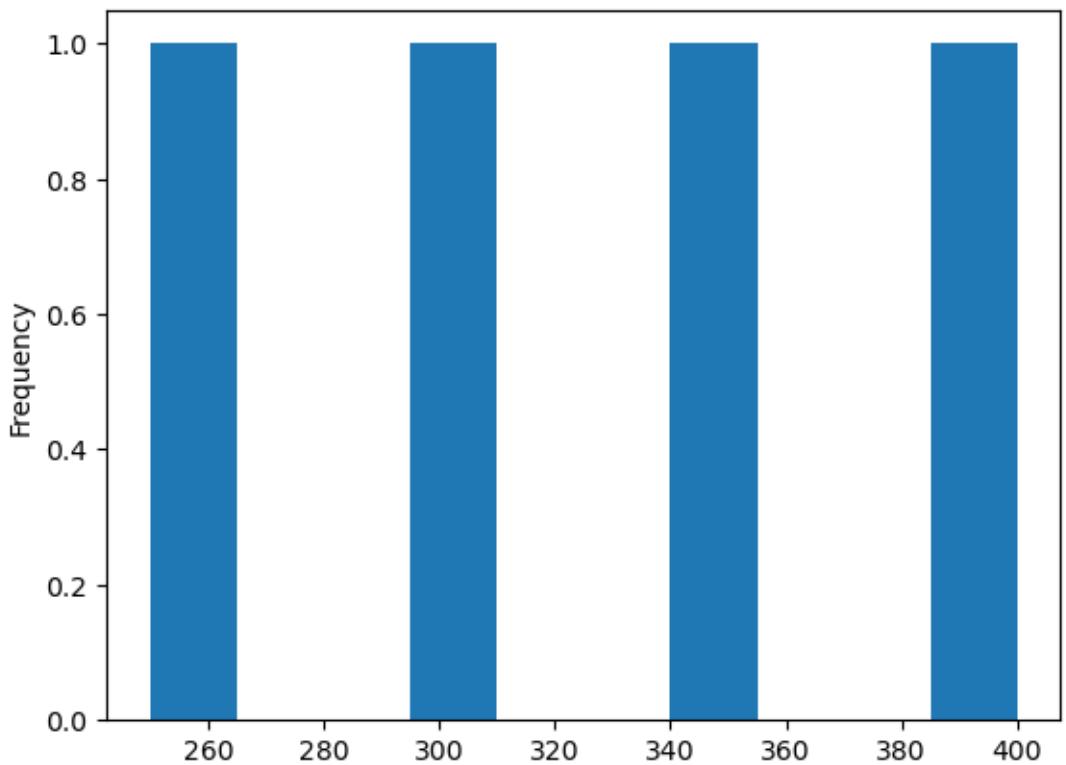
[34]:
```
# Area plot
df.plot(x='month', y='sales', kind='area');
```



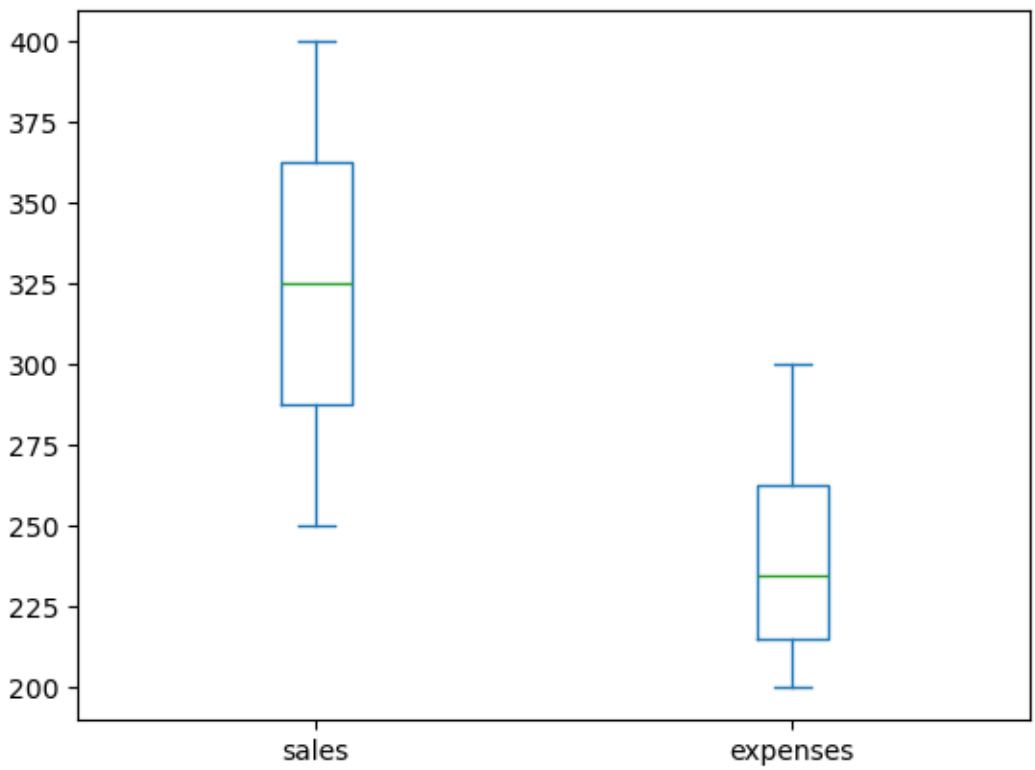
[35]:
```
# Scatter plot
df.plot(x='month', y='sales', kind='scatter');
```



[37]:
```
# hist plot
# df.plot(x='sales', kind='hist');
df['sales'].plot(kind='hist');
```



[39]:
```
# box plot
# df['sales'].plot(kind='box');
df[['sales','expenses']].plot(kind='box');
```


400
375
350
325
300
275
250
225
200
sales
expenses

[ ]:

```
```